SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	S Susmitha	**Reg. No.:**	192571068
Section / Batch:	Slot B	**Date:**	05.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I **S Susmitha (192571068)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____  _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA0603	Design and Analysis of Algorithms

CASE STUDY:
ENGINEERING SIMULATION SYSTEM
(MATRIX MULTIPLICATION)

Question:

An engineering simulation tool performs repeated matrix operations for structural analysis. Analyze how divide and conquer matrix multiplication improves computational efficiency. Identify challenges in large-scale matrix handling. Apply optimized methods such as Strassen's algorithm. Analyze performance and design a solution, justifying its advantages.

Description of the Real-World Problem

Engineering simulation systems used for structural analysis, such as Finite Element Analysis (FEA) software, rely on large-scale numerical modeling of physical structures like bridges, buildings, aircraft frames, and mechanical assemblies. These simulations represent structural properties — stiffness, stress, strain, and displacement — as large matrices, and computing the response of a structure under load requires repeated multiplication of these matrices.

As the mesh resolution of a simulation increases (more nodes and elements used to model the structure), the matrices involved grow substantially in size, sometimes running into thousands of rows and columns. Since matrix multiplication is one of the most frequently executed operations in such simulations, even small inefficiencies in the multiplication algorithm translate into significant increases in computation time, hardware cost, and energy consumption across a full simulation run.

Problem Statement

Given two large matrices A and B representing structural sub-blocks (e.g., stiffness or transformation matrices) in an engineering simulation, design an algorithm to compute the product matrix $C = A \times B$ that minimizes computational time compared to the standard $O(n^3)$ approach, while remaining numerically reliable and suitable for large, repeated use within a simulation pipeline.

Algorithm Used

Strassen's Matrix Multiplication Algorithm — a divide and conquer algorithm that reduces the number of recursive multiplications needed to multiply two n × n matrices from 8 (in the naive divide and conquer method) to 7, by cleverly combining matrix additions and subtractions.

Core idea: Split each matrix into four n/2 × n/2 submatrices, compute 7 intermediate products (M1–M7) using combinations of these submatrices, and reconstruct the four quadrants of the result matrix using only additions and subtractions of these 7 products.

The seven products computed are:

M1 = (A11 + A22)(B11 + B22)
M2 = (A21 + A22) B11
M3 = A11 (B12 − B22)
M4 = A22 (B21 − B11)
M5 = (A11 + A12) B22
M6 = (A21 − A11)(B11 + B12)
M7 = (A12 − A22)(B21 + B22)

The result quadrants are then assembled as:

C11 = M1 + M4 − M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 − M2 + M3 + M6

Justification for Choosing the Algorithm (Why This Algorithm?)

- Reduces the number of recursive multiplications per split from 8 to 7, giving a genuine asymptotic improvement from $O(n^3)$ to O(n^2.807).
- Matrix multiplication is a repeated, high-frequency operation in structural simulations, so even a modest asymptotic improvement compounds into large real-world time savings across thousands of iterations.
- The recursive structure naturally decomposes into independent subproblems, which map well onto multi-core and distributed simulation clusters.
- Compared to more advanced sub-cubic algorithms (e.g., Coppersmith–Winograd family), Strassen's algorithm has practically usable constant factors, making it implementable in real engineering software rather than remaining a theoretical result.
- It can be combined with a hybrid strategy (switching to standard multiplication below a threshold size) to avoid recursion overhead on small matrices, making it practical across the full range of matrix sizes seen in simulation.

Complexity Analysis

Naive Divide and Conquer

Splitting each matrix into 4 submatrices and performing 8 recursive multiplications plus $O(n^2)$ additions gives the recurrence:

$T(n) = 8T(n/2) + O(n^2) \Rightarrow T(n) = O(n^3)$ (by Master Theorem)

Strassen's Algorithm

By reducing the recursive multiplications to 7 (at the cost of more $O(n^2)$ additions/subtractions), the recurrence becomes:

$T(n) = 7T(n/2) + O(n^2) \Rightarrow T(n) = O(n^{\log_2 7}) \approx O(n^{2.807})$

Aspect	Naive Multiplication	Strassen's Algorithm
Recursive multiplications	8	7
Recurrence relation	$T(n) = 8T(n/2) + O(n^2)$	$T(n) = 7T(n/2) + O(n^2)$
Time Complexity	$O(n^3)$	$O(n^{\log_2 7}) \approx O(n^{2.807})$
Space Complexity	$O(n^2)$	$O(n^2)$ (extra temp matrices)
Additions/Subtractions	Fewer	More (18 vs 4)

Space complexity remains $O(n^2)$, but Strassen's algorithm requires additional temporary matrices during recursion (for the M1–M7 products and intermediate sums), which increases the constant factor of memory usage compared to the naive approach.

Manual Execution (Step-by-Step Process)

Consider two 4 × 4 structural sub-block matrices:

```
A = [ 1  2  3  4 ]      B = [16 15 14 13]
    [ 5  6  7  8 ]          [12 11 10  9]
    [ 9 10 11 12]           [ 8  7  6  5]
    [13 14 15 16]           [ 4  3  2  1]
```

Step 1 – Divide: Split A and B into four 2×2 submatrices each (A11, A12, A21, A22 and B11, B12, B21, B22).

Step 2 – Compute the 7 products: Using the base-case 2×2 blocks, compute M1 through M7 using the formulas defined in the Algorithm Used section, applying standard multiplication at this small base size.

Step 3 – Combine: Assemble C11, C12, C21, C22 from M1–M7 using only additions and subtractions.

Step 4 – Reassemble: Merge the four quadrants (C11, C12, C21, C22) back into the full 4 × 4 result matrix C.

Final result obtained (verified by program execution below):

```
C = [ 80   70   60   50]
    [240  214  188  162]
    [400  358  316  274]
    [560  502  444  386]
```

Applications

- Finite Element Analysis (FEA) for structural, thermal, and fluid simulations in civil and mechanical engineering.
- Computer graphics — 3D transformations, rendering pipelines, and physics engines.
- Scientific computing — solving large systems of linear equations, eigenvalue problems.
- Machine learning — training and inference in neural networks, which are dominated by matrix multiplications.
- Robotics — kinematic and dynamic modeling using transformation matrices.
- Cryptography and signal processing — operations on large data matrices.

Advantages

- Asymptotically faster than the naive $O(n^3)$ method: $O(n^{2.807})$ vs $O(n^3)$.
- Performance gains increase as matrix size grows, making it well-suited to large-scale simulations.
- Recursive, independent subproblems parallelize naturally across multiple cores or nodes.
- Improves cache locality by operating on smaller submatrices.
- Serves as the conceptual basis for further optimized algorithms.

Limitations

- Higher constant factor and recursion/memory overhead makes it slower than naive multiplication for small matrices.
- Requires matrix dimensions to be (or be padded to) powers of two, wasting computation on non-uniform sizes.
- Increased number of additions/subtractions can amplify floating-point rounding errors, a concern for precision-sensitive engineering computations.
- Does not exploit sparsity — inefficient for the sparse stiffness matrices common in real FEA models unless combined with sparse-matrix techniques.
- More complex to implement and debug than the straightforward triple-loop method.

Comparison with Other Algorithms

Algorithm	**Time Complexity**	**Approach**	**Best Use Case**
Naive (Standard) Multiplication	$O(n^3)$	Three nested loops	Small matrices, simplicity
Basic Divide & Conquer	$O(n^3)$	8 recursive sub-multiplications	Parallelization, cache locality
Strassen's Algorithm	$O(n^{2.807})$	7 recursive sub-multiplications	Large dense matrices
Coppersmith–Winograd (theoretical)	$O(n^{2.373})$	Advanced tensor methods	Theoretical interest only – huge constants

Screenshot of the Program

```c
#include <stdio.h>
#include <stdlib.h>

#define N 4  // matrix size (must be power of 2 for this demo)

void printMatrix(int **M, int n, const char *label) {
    printf("%s:\n", label);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%6d ", M[i][j]);
        printf("\n");
    }
    printf("\n");
}

int** allocMatrix(int n) {
    int **M = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++)
        M[i] = (int*)calloc(n, sizeof(int));
    return M;
}
```

```
23  void freeMatrix(int **M, int n) {
24      for (int i = 0; i < n; i++) free(M[i]);
25      free(M);
26  }
27
28  void add(int **A, int **B, int **C, int n) {
29      for (int i = 0; i < n; i++)
30          for (int j = 0; j < n; j++)
31              C[i][j] = A[i][j] + B[i][j];
32  }
33
34  void sub(int **A, int **B, int **C, int n) {
35      for (int i = 0; i < n; i++)
36          for (int j = 0; j < n; j++)
37              C[i][j] = A[i][j] - B[i][j];
38  }
39
40  // Standard multiplication used as base case when n is small
41  void standardMultiply(int **A, int **B, int **C, int n) {
42      for (int i = 0; i < n; i++)
43          for (int j = 0; j < n; j++) {
44              C[i][j] = 0;
45              for (int k = 0; k < n; k++)
```

```
41  void standardMultiply(int **A, int **B, int **C, int n) {
43          for (int j = 0; j < n; j++) {
46              C[i][j] += A[i][k] * B[k][j];
47          }
48  }
49
50  void strassen(int **A, int **B, int **C, int n) {
51      if (n <= 2) {  // base case threshold
52          standardMultiply(A, B, C, n);
53          return;
54      }
55
56      int newSize = n / 2;
57      int **A11 = allocMatrix(newSize), **A12 = allocMatrix(newSize);
58      int **A21 = allocMatrix(newSize), **A22 = allocMatrix(newSize);
59      int **B11 = allocMatrix(newSize), **B12 = allocMatrix(newSize);
60      int **B21 = allocMatrix(newSize), **B22 = allocMatrix(newSize);
61
62      for (int i = 0; i < newSize; i++)
63          for (int j = 0; j < newSize; j++) {
64              A11[i][j] = A[i][j];
65              A12[i][j] = A[i][j + newSize];
66              A21[i][j] = A[i + newSize][j];
```

```
50  void strassen(int **A, int **B, int **C, int n) {
63          for (int j = 0; j < newSize; j++) {
67              A22[i][j] = A[i + newSize][j + newSize];
68              B11[i][j] = B[i][j];
69              B12[i][j] = B[i][j + newSize];
70              B21[i][j] = B[i + newSize][j];
71              B22[i][j] = B[i + newSize][j + newSize];
72          }
73
74      int **M1 = allocMatrix(newSize), **M2 = allocMatrix(newSize), **M3 = allocMatrix(newSize);
75      int **M4 = allocMatrix(newSize), **M5 = allocMatrix(newSize), **M6 = allocMatrix(newSize);
76      int **M7 = allocMatrix(newSize);
77      int **T1 = allocMatrix(newSize), **T2 = allocMatrix(newSize);
78
79      add(A11, A22, T1, newSize); add(B11, B22, T2, newSize); strassen(T1, T2, M1, newSize);
80      add(A21, A22, T1, newSize); strassen(T1, B11, M2, newSize);
81      sub(B12, B22, T2, newSize); strassen(A11, T2, M3, newSize);
82      sub(B21, B11, T2, newSize); strassen(A22, T2, M4, newSize);
83      add(A11, A12, T1, newSize); strassen(T1, B22, M5, newSize);
84      sub(A21, A11, T1, newSize); add(B11, B12, T2, newSize); strassen(T1, T2, M6, newSize);
85      sub(A12, A22, T1, newSize); add(B21, B22, T2, newSize); strassen(T1, T2, M7, newSize);
```

```
    int **C11 = allocMatrix(newSize), **C12 = allocMatrix(newSize);
    int **C21 = allocMatrix(newSize), **C22 = allocMatrix(newSize);

    add(M1, M4, T1, newSize); sub(T1, M5, T2, newSize); add(T2, M7, C11, newSize);
    add(M3, M5, C12, newSize);
    add(M2, M4, C21, newSize);
    sub(M1, M2, T1, newSize); add(T1, M3, T2, newSize); add(T2, M6, C22, newSize);

    for (int i = 0; i < newSize; i++)
        for (int j = 0; j < newSize; j++) {
            C[i][j] = C11[i][j];
            C[i][j + newSize] = C12[i][j];
            C[i + newSize][j] = C21[i][j];
            C[i + newSize][j + newSize] = C22[i][j];
        }

    freeMatrix(A11,newSize); freeMatrix(A12,newSize); freeMatrix(A21,newSize); freeMatrix(A22,newSize
    freeMatrix(B11,newSize); freeMatrix(B12,newSize); freeMatrix(B21,newSize); freeMatrix(B22,newSize
    freeMatrix(M1,newSize); freeMatrix(M2,newSize); freeMatrix(M3,newSize); freeMatrix(M4,newSize);
    freeMatrix(M5,newSize); freeMatrix(M6,newSize); freeMatrix(M7,newSize);
    freeMatrix(T1,newSize); freeMatrix(T2,newSize);
    freeMatrix(C11,newSize); freeMatrix(C12,newSize); freeMatrix(C21,newSize); freeMatrix(C22,newSize
```

```
int main() {
    int **A = allocMatrix(N);
    int **B = allocMatrix(N);
    int **C = allocMatrix(N);

    int a[N][N] = {{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};
    int b[N][N] = {{16,15,14,13},{12,11,10,9},{8,7,6,5},{4,3,2,1}};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = a[i][j];
            B[i][j] = b[i][j];
        }

    printf("=== Strassen's Matrix Multiplication (Divide and Conquer) ===\n\n");
    printMatrix(A, N, "Matrix A (Structural Stiffness Sub-Block 1)");
    printMatrix(B, N, "Matrix B (Structural Stiffness Sub-Block 2)");

    strassen(A, B, C, N);

    printMatrix(C, N, "Result C = A x B (via Strassen's Algorithm)");
    printf("Multiplications performed: 7 per split level (vs 8 in naive divide & conquer)\n");
    printf("Time Complexity: O(n^log2(7)) ~ O(n^2.807)\n");
```

```c
    int a[N][N] = {{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};
    int b[N][N] = {{16,15,14,13},{12,11,10,9},{8,7,6,5},{4,3,2,1}};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = a[i][j];
            B[i][j] = b[i][j];
        }

    printf("=== Strassen's Matrix Multiplication (Divide and Conquer) ===\n\n");
    printMatrix(A, N, "Matrix A (Structural Stiffness Sub-Block 1)");
    printMatrix(B, N, "Matrix B (Structural Stiffness Sub-Block 2)");

    strassen(A, B, C, N);

    printMatrix(C, N, "Result C = A x B (via Strassen's Algorithm)");
    printf("Multiplications performed: 7 per split level (vs 8 in naive divide & conquer)\n");
    printf("Time Complexity: O(n^log2(7)) ~ O(n^2.807)\n");

    freeMatrix(A, N); freeMatrix(B, N); freeMatrix(C, N);
    return 0;
}
```

Fig 1: C implementation of Strassen's Matrix Multiplication Algorithm

Output Screenshot

```
PS C:\Users\Susmitha\Downloads\DAA Code Implementation> gcc CO3_AT2.c -o CO3_AT2.exe
PS C:\Users\Susmitha\Downloads\DAA Code Implementation> .\CO3_AT2.exe
=== Strassen's Matrix Multiplication (Divide and Conquer) ===

Matrix A (Structural Stiffness Sub-Block 1):
     1      2      3      4
     5      6      7      8
     9     10     11     12
    13     14     15     16

Matrix B (Structural Stiffness Sub-Block 2):
    16     15     14     13
    12     11     10      9
     8      7      6      5
     4      3      2      1

Result C = A x B (via Strassen's Algorithm):
    80     70     60     50
   240    214    188    162
   400    358    316    274
   560    502    444    386

Multiplications performed: 7 per split level (vs 8 in naive divide & conquer)
Time Complexity: O(n^log2(7)) ~ O(n^2.807)
```

Fig 2: Program output showing input matrices and the multiplication result

Conclusion

Strassen's divide-and-conquer algorithm reduces matrix multiplication complexity from $O(n^3)$ to $O(n^{2.807})$, enabling faster large-scale engineering simulations and improved hardware utilization. Although it has higher memory usage and is less effective for small or sparse matrices, these limitations can be addressed using hybrid multiplication, matrix padding, sparse techniques, and parallel execution. This makes Strassen's algorithm a practical optimization for engineering simulation systems.